

Competitive Programming

Section 31 Editorial: Sieve, Primes, and Factorization

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Prime Counter	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Factor Signature	7
3.1	Restatement	7
3.2	Algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	8
4	C. Divisor Queries	10
4.1	Restatement	10
4.2	Algorithm	10
4.3	Correctness proof	10
4.4	Full C++ solution	10
4.5	Full Python solution	11
5	Testing discussion	12
6	What to carry forward	13

1 Editorial overview

Problem	Preprocessing	Query
A. Prime Counter	prime sieve + prefix	range prime count
B. Factor Signature	smallest prime factor	factor statistics
C. Divisor Queries	smallest prime factor	divisor count

2 A. Prime Counter

2.1 Restatement

Count primes in many intervals $[l, r]$.

2.2 Algorithm

Build `is_prime` with the sieve of Eratosthenes, then build:

```
pref[i] = pref[i-1] + is_prime[i]
```

Answer each query by `pref[r] - pref[l-1]`.

2.3 Correctness proof

The sieve marks a composite number when processing one of its prime divisors. Every composite x has a prime divisor at most $\sqrt{x}$, so it is marked. No prime is ever marked, because a prime has no smaller divisor. Thus `is_prime` is correct.

The prefix array counts primes up to each position, so subtracting counts before l leaves exactly the primes in $[l, r]$. $\square$

2.4 Complexity

```
Preprocessing:  $O(n \log \log n)$   
Each query:    $O(1)$   
Memory:        $O(n)$ 
```

2.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
vector<bool> is_prime(n + 1, true);
if (n >= 0) is_prime[0] = false;
if (n >= 1) is_prime[1] = false;
for (long long p = 2; p * p <= n; p++) {
    if (is_prime[p]) {
        for (long long x = p * p; x <= n; x += p) is_prime[x] = false;
    }
}
vector<int> pref(n + 1, 0);
for (int i = 1; i <= n; i++) pref[i] = pref[i - 1] + is_prime[i];
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << pref[r] - pref[l - 1] << '\n';
}
return 0;
}

```

2.6 Full Python solution

```

import sys

def sieve(n: int) -> list[bool]:
    is_prime = [True] * (n + 1)
    if n >= 0:
        is_prime[0] = False
    if n >= 1:
        is_prime[1] = False
    p = 2
    while p * p <= n:
        if is_prime[p]:
            for x in range(p * p, n + 1, p):
                is_prime[x] = False
        p += 1
    return is_prime

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    is_prime = sieve(n)
    pref = [0] * (n + 1)
    for i in range(1, n + 1):

```

```
        pref[i] = pref[i - 1] + int(is_prime[i])
    idx = 2
    out = []
    for _ in range(q):
        l, r = data[idx], data[idx + 1]
        idx += 2
        out.append(str(pref[r] - pref[l - 1]))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

3 B. Factor Signature

3.1 Restatement

For each x , print the number of distinct prime factors, the total number of prime factors with multiplicity, and the largest prime factor.

3.2 Algorithm

Build `spf` up to the maximum query value. For each x , repeatedly take $p = \text{spf}[x]$, count how many times it divides x , and divide it out.

3.3 Correctness proof

`spf[x]` is a prime divisor of x , and specifically the smallest one. Dividing out all copies of `spf[x]` records exactly one distinct factor and its exponent. Repeating until $x=1$ records every prime-power block in the unique prime factorization. The requested statistics are direct sums over those blocks. $\square$

3.4 Complexity

```
Preprocessing:  $O(N \log \log N)$   
Each query:  $O(\text{number of prime factors})$ 
```

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    int mx = 1;
```

```

for (int &x : values) {
    cin >> x;
    mx = max(mx, x);
}
vector<int> spf(mx + 1);
iota(spf.begin(), spf.end(), 0);
if (mx >= 1) spf[1] = 1;
for (long long p = 2; p <= mx; p++) {
    if (spf[p] == p && p * p <= mx) {
        for (long long x = p * p; x <= mx; x += p) {
            if (spf[x] == x) spf[x] = p;
        }
    }
}
for (int x0 : values) {
    if (x0 == 1) {
        cout << "0 0 1\n";
        continue;
    }
    int x = x0, distinct = 0, total = 0, largest = 1;
    while (x > 1) {
        int p = spf[x];
        distinct++;
        largest = p;
        while (x % p == 0) {
            total++;
            x /= p;
        }
    }
    cout << distinct << ' ' << total << ' ' << largest << '\n';
}
return 0;
}

```

3.6 Full Python solution

```

import sys

def spf_table(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, n + 1):
        if spf[p] == p and p * p <= n:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

```

The full reference file in the section uses this table and the repeated division loop described above.

4 C. Divisor Queries

4.1 Restatement

For each x , print the number of positive divisors.

4.2 Algorithm

Factor with SPF. If:

$$x = p_1^{e_1} * p_2^{e_2} * \dots * p_k^{e_k}$$

then:

$$\tau(x) = (e_1+1)(e_2+1)\dots(e_k+1)$$

4.3 Correctness proof

Every divisor chooses an exponent for each prime factor independently. For prime p_i , there are e_i+1 choices: $0, 1, \dots, e_i$. Multiplying the independent choice counts gives the formula. SPF factorization gives the correct exponents, so the computed product is correct. $\square$

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    int mx = 1;
    for (int &x : values) {
        cin >> x;
        mx = max(mx, x);
    }
```

```

vector<int> spf(mx + 1);
iota(spf.begin(), spf.end(), 0);
if (mx >= 1) spf[1] = 1;
for (long long p = 2; p <= mx; p++) {
    if (spf[p] == p && p * p <= mx) {
        for (long long x = p * p; x <= mx; x += p) {
            if (spf[x] == x) spf[x] = p;
        }
    }
}
for (int x : values) {
    int ans = 1;
    while (x > 1) {
        int p = spf[x];
        int exp = 0;
        while (x % p == 0) {
            exp++;
            x /= p;
        }
        ans *= exp + 1;
    }
    cout << ans << '\n';
}
return 0;
}

```

4.5 Full Python solution

```

import sys

def spf_table(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, n + 1):
        if spf[p] == p and p * p <= n:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

```

Then divide by `spf[x]`, count the exponent, and multiply the answer by `exponent + 1`.

5 Testing discussion

The random tests compare the optimized SPF solutions against slow trial division for small values. Important fixed tests include:

```
n = 1
x = 1
large prime near the limit
prime powers such as 64
numbers with several distinct primes such as 210
```

6 What to carry forward

Use primality sieves for prime flags and prefix counts. Use SPF tables when you must factor many values. Once factorized, many divisor properties become simple formulas over prime exponents.